

Projeto 1 - Jogo “Adivinhe o número” implementado em RISC-V

Disciplina: SSC0902 - Organização e Arquitetura de Computadores

Ariel Alves da Silva

8847378

São Carlos, SP

2026

Sumário

1	Introdução	3
2	Desenvolvimento do Jogo	3
2.1	Seção de dados	3
2.2	Chamadas de funções	3
2.3	Funções	4
2.4	Gerador de números aleatórios	4
2.5	Lista encadeada	5
2.6	Reset de registradores	5
3	Execução do programa	5
4	Desafios enfrentados	6

1 Introdução

O trabalho consiste em implementar um jogo simples, “Adivinhe o Número”. Os requisitos do trabalho são:

- Exibir mensagem de boas vindas e instruções para o jogador
- Geração de número aleatório com algoritmo congruente linear entre 1 e 100
- Jogador deve tentar adivinhar o número. Após cada palpite, o jogo fornecerá feedback indicando se o palpite é muito alto, muito baixo ou correto.
- Armazenamento das tentativas em lista ligada dinamicamente (na heap)
- Encerramento do jogo quando o jogador adivinhar o número, exibindo uma mensagem de parabéns, o número de tentativas feitas e todas as tentativas, percorrendo a lista ligada até o fim.

2 Desenvolvimento do Jogo

O jogo é simples, e o desenvolvimento exigiu apenas a consulta de comandos RISC-V associados às instruções necessárias para as operações requisitadas pelo projeto.

Serão especificados adiante os detalhes concernentes à implementação do código.

2.1 Seção de dados

Na seção de dados, foram inseridas as declarações de strings, principalmente as mensagens que serão mostradas ao jogador, e as variáveis globais que armazenarão a cabeça e cauda da lista encadeada. As strings possuem alinhamento de 1 byte, enquanto os endereços das listas possuem alinhamento de 4 bytes.

```
.data
.align 0
welcome_message: .asciz "Welcome to 'guess the number'! You must guess a randomly
generated integer number between 1 and 100. \n\nAfter each guessing, you will receive a
feedback (guessing it greater than, less than, or correct). Good luck!\n"
input_message: .asciz "\nWrite a number [1,100]: "
lt_message: .asciz "Less than generated number.\n"
gt_message: .asciz "\nGreater than generated number."
eq_message: .asciz "\nCongrats! You guessed it."
n_attempts_message: .asciz "\nNumber of attempts: "
attempts_message: .asciz "\nAttempts: "
arrow_str: .asciz " -> "
end_str: .asciz "END\n"
.align 2
lkd_list_head: .space 4
lkd_list_tail: .space 4
```

2.2 Chamadas de funções

As chamadas de funções seguem um padrão. A Chamada da função é feita com

`jal ra, function`. Já dentro da função, os valores dos parâmetros (a0, a1, ...) são armazenados na stack para uso posterior (stack frame), além do valor de retorno colocado no `ra`, de modo que a stack fica igual a estrutura abaixo:

```

param_n <- SP
-----
...
-----
param_2
-----
param_1
-----
ret

```

Essa estrutura foi criada pois há chamadas de funções dentro de outras funções. Assim, pode-se recuperar as variáveis dadas ao loop, por exemplo, simplesmente destruindo stack frames.

Para construção do stack frame, faz-se, por exemplo, `addi sp, sp, -12`, posteriormente armazenando as variáveis baseando-se em `sp`. Para destruição do stack frame, faz-se `addi sp, sp, 8` e recupera-se o valor de retorno colocando-o em `ra`, posteriormente fazendo-se `addi sp, sp, 4` e `ret`.

Uma observação é que essa estrutura somente foi aplicada a funções onde ela seja necessária, ou seja, recebimento de parâmetros e chamadas internas de outras funções. Em funções que simplesmente retornam um valor (como gerar um número aleatório) ou imprimem algo na tela (como mostrar as tentativas), não há uso de stack frame, ou seja, há apenas a chamada por `jal ra, function`, o código, e posteriormente o `ret`.

2.3 Funções

- `gen_rand_int` - Função para gerar número aleatório utilizando o tempo atual em milissegundos. Nenhum parâmetro. Retorna um número aleatório em `a0`.
- `loop` - Função para receber e tratar inputs do usuário. Recebe o número aleatório em `a0`. Não retorna nada.
- `verify_number` - Função para verificar se número do usuário é maior, menor ou igual ao número aleatório. Recebe o número do usuário em `a0` e o número aleatório em `a1`. Retorna um valor em `a0`: 0 para `a0 == a1`, 1 para `a0 < a1`, 2 para `a0 > a1`.
- `add_attempt` - Função para adicionar a tentativa à lista encadeada. Recebe o número do usuário em `a0`.
- `show_attempts` - Função para percorrer a lista encadeada e mostrar os valores inseridos, até encontrar um ponteiro para a próxima igual a nulo (preenchido de zeros).
- `end` - Encerra o programa com a syscall Exit. (código 10)

2.4 Gerador de números aleatórios

Para gerar números aleatórios, foi utilizado um algoritmo congruente linear:

$$n = (a \cdot x + b) \bmod m$$

Com:

- n = número aleatório gerado
- a = multiplicador $0 < a < m$
- c = incremento $0 \leq c < m$

- $m = \text{módulo } 0 < m$
- $x = \text{semente } 0 < x < m$

A semente utilizada foi o tempo atual, em milissegundos, obtido por meio de uma syscall. Os parâmetros foram definidos arbitrariamente como sendo $a = 1$, $c = 5$, $m = 100$. Isso gera um número aleatório único entre 0 e 100 que será utilizado pelo programa posteriormente. Como o valor mínimo é 1, foi implementado um loop que gera o número aleatório quantas vezes for necessário, até obter um número diferente de zero.

2.5 Lista encadeada

A lista encadeada consiste em blocos de 8 bytes `[user_number][next_addr]`, onde o primeiro bloco de 4 bytes corresponde ao número do usuário, e o segundo bloco corresponde ao endereço do próximo bloco da lista encadeada. O endereço do próximo é preenchido de zeros no caso onde não há próximo na lista. A lista é inteiramente alocada na heap.

2.6 Reset de registradores

Para o reset de registradores, (zerar), foi aplicada a operação `xor` do registrador com ele mesmo.

3 Execução do programa

Segue abaixo uma execução padrão do programa por meio do simulador RARS.

```
Welcome to 'guess the number'! You must guess a randomly generated integer number
between 1 and 100.

After each guessing, you will receive a feedback (guessing it greater than, less than,
or correct). Good luck!

Write a number [1,100]: 1
Less than generated number.

Write a number [1,100]: 100

Greater than generated number.
Write a number [1,100]: 2
Less than generated number.

Write a number [1,100]: 99

Greater than generated number.
Write a number [1,100]: 33

Greater than generated number.
Write a number [1,100]: 66

Greater than generated number.
Write a number [1,100]: 20

Greater than generated number.
Write a number [1,100]: 15

Greater than generated number.
Write a number [1,100]: 12
```

```
Greater than generated number.  
Write a number [1,100]: 10
```

```
Greater than generated number.  
Write a number [1,100]: 9
```

```
Greater than generated number.  
Write a number [1,100]: 8
```

```
Greater than generated number.  
Write a number [1,100]: 5  
Less than generated number.
```

```
Write a number [1,100]: 6  
Less than generated number.
```

```
Write a number [1,100]: 7
```

```
Congrats! You guessed it.
```

```
Number of attempts: 15
```

```
Attempts: 1 -> 100 -> 2 -> 99 -> 33 -> 66 -> 20 -> 15 -> 12 -> 10 -> 9 -> 8 -> 5 -> 6 ->  
7 -> END
```

4 Desafios enfrentados

O projeto foi implementado de maneira simples, com problemas enfrentados relacionados à falta de atenção (digitar o nome de um registrador errado ou um comando errado).